Name of Student: Husam Nour	**PRN:25070122086**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	

ASSIGNMENT No: 25

TITLE: Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }
    public void run() {
        System.out.println(getName() + " is running.");
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println(getName() + " was interrupted.");
        }
        System.out.println(getName() + " has finished.");
    }
}
public class A25 { public static void main(String[] args) throws
    InterruptedException {
        MyThread t1 = new MyThread("Thread 1");
        MyThread t2 = new MyThread("Thread 2");
        MyThread t3 = new MyThread("Thread 3");
        // NEW state
        System.out.println("Thread 1 state: " + t1.getState());
        System.out.println("Thread 2 state: " + t2.getState());
        System.out.println("Thread 3 state: " + t3.getState());
        // Start all threads
        t1.start();
        t2.start();
        t3.start();
        // Wait for all threads to
        finish t1.join(); t2.join();
        t3.join();
        // TERMINATED state
        System.out.println("\nThread 1 state: " + t1.getState());
        System.out.println("Thread 2 state: " + t2.getState());
```

```
        System.out.println("Thread 3 state: " + t3.getState());
        System.out.println("All threads have completed.");
    }
}
```

Learning Objectives

To create and manage multiple threads using the Thread class.

Programme

1. Create a program where multiple threads print numbers simultaneously

```
class NumberThread extends Thread {
    NumberThread(String name) {
        super(name);
    }
    public void run() { for (int i =
        1; i <= 5; i++) {
            System.out.println(getName() + " : " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class A25 { public static void
    main(String[] args) {
        NumberThread t1 = new NumberThread("Thread 1");
        NumberThread t2 = new NumberThread("Thread
        2"); NumberThread t3 = new
        NumberThread("Thread 3"); t1.start();
        t2.start(); t3.start();
    }
}
```

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
class BankAccount {
    int balance =
    1000;
    synchronized void deposit(int amount) {
        balance += amount;
```

```
        System.out.println(Thread.currentThread().getName()
                + " deposited: Rs. " + amount);
        System.out.println("Current Balance: Rs. " + balance);
    }
    synchronized void withdraw(int amount)
        { if (balance >= amount) { balance
        -= amount;
            System.out.println(Thread.currentThread().getName()
                    + " withdrew: Rs. " + amount);
            System.out.println("Current Balance: Rs. " + balance);
        } else {
            System.out.println(Thread.currentThread().getName()
                    + " cannot withdraw. Insufficient balance!");
        }
    }
}
class DepositThread extends Thread {
    BankAccount account;
    DepositThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.deposit(500);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
class WithdrawThread extends Thread {
    BankAccount account;
    WithdrawThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.withdraw(300);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class A25 { public static void
    main(String[] args) {
        BankAccount account = new BankAccount();
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new
```

```
        WithdrawThread(account); t1.setName("Deposit
        Thread"); t2.setName("Withdrawal Thread");
        t1.start();
```

```
        t2.start();
    }
}
```

Output

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

```
PS C:\CS-JAVA\28> javac A25.java
PS C:\CS-JAVA\28> java A25
Thread 1 state: NEW
Thread 2 state: NEW
Thread 3 state: NEW
Thread 1 is running.
Thread 2 is running.
Thread 3 is running.
Thread 2 has finished.
Thread 3 has finished.
Thread 1 has finished.

Thread 1 state: TERMINATED
Thread 2 state: TERMINATED
Thread 3 state: TERMINATED
All threads have completed.
PS C:\CS-JAVA\28>
```

1.Create a program where multiple threads print numbers simultaneously

```
PS C:\CS-JAVA\28> javac A25.java
PS C:\CS-JAVA\28> java A25
Thread 2 : 1
Thread 1 : 1
Thread 3 : 1
Thread 2 : 2
Thread 1 : 2
Thread 3 : 2
Thread 2 : 3
Thread 1 : 3
Thread 3 : 3
Thread 2 : 4
Thread 1 : 4
Thread 3 : 4
Thread 2 : 5
Thread 1 : 5
Thread 3 : 5
PS C:\CS-JAVA\28>
```

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
PS C:\CS-JAVA\28> javac A25.java
PS C:\CS-JAVA\28> java A25
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1500
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1200
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1700
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1400
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1900
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1600
PS C:\CS-JAVA\28>
```

1.Create a program where multiple threads print numbers simultaneously

```
class NumberThread extends Thread {
    NumberThread(String name) {
        super(name);
    }
    public void run() { for (int i =
        1; i <= 5; i++) {
            System.out.println(getName() + " prints: " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class A25 { public static void
    main(String[] args) {
        NumberThread t1 = new NumberThread("Thread 1");
        NumberThread t2 = new NumberThread("Thread
        2"); NumberThread t3 = new
        NumberThread("Thread 3"); t1.start();
        t2.start(); t3.start();
    }
}
```

```
PS C:\CS-JAVA\28> javac A25.java
PS C:\CS-JAVA\28> java A25
Thread 1 prints: 1
Thread 3 prints: 1
Thread 2 prints: 1
Thread 1 prints: 2
Thread 3 prints: 2
Thread 2 prints: 2
Thread 1 prints: 3
Thread 3 prints: 3
Thread 2 prints: 3
Thread 3 prints: 4
Thread 2 prints: 4
Thread 1 prints: 4
Thread 3 prints: 5
Thread 2 prints: 5
Thread 1 prints: 5
PS C:\CS-JAVA\28>
```

2.Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
class BankAccount {
    int balance =
    1000;
```

```
    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(Thread.currentThread().getName()
                + " deposited: Rs. " + amount);
        System.out.println("Current Balance: Rs. " + balance);
    }
    synchronized void withdraw(int amount)
        { if (balance >= amount) { balance
        -= amount;
            System.out.println(Thread.currentThread().getName()
                    + " withdrew: Rs. " + amount);
            System.out.println("Current Balance: Rs. " + balance);
        } else {
            System.out.println(Thread.currentThread().getName()
                    + " cannot withdraw. Insufficient balance!");
        }
    }
}
class DepositThread extends Thread {
    BankAccount account;
    DepositThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
```

```
            account.deposit(500);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
class WithdrawThread extends Thread {
    BankAccount account;
    WithdrawThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.withdraw(300);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class A25 { public static void
    main(String[] args) {
        BankAccount account = new BankAccount();
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new
        WithdrawThread(account); t1.setName("Deposit
        Thread"); t2.setName("Withdrawal Thread");
        t1.start(); t2.start();
    }
}
```

```
PS C:\CS-JAVA\28> javac A25.java
PS C:\CS-JAVA\28> java A25
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1500
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1200
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 900
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1400
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1100
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1600
PS C:\CS-JAVA\28>
```

Conclusion

In conclusion, threads in Java allow multiple tasks to execute concurrently, improving the performance and responsiveness of a program. The Thread class provides methods such as start(), run(), and sleep() to manage thread execution. Multithreading is useful in applications such as banking systems, ticket booking, and online shopping, where multiple operations occur at the same time. The synchronized keyword helps prevent data inconsistency when multiple threads access shared resources.

A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.